

```

/**
 * server/index.ts – PATCH
 *
 * Wire the three new services into the existing startup sequence.
 * Add these imports and startup calls to your existing server/index.ts.
 */

// — 1. Add imports (alongside your existing service imports) —————

import { startShortsPrepPipeline, stopShortsPrepPipeline } from './services/shorts-prep-pipeline.js';
import { startLongformPrepPipeline, stopLongformPrepPipeline } from './services/longform-prep-pipeline.js';
import { startQuotaAwarePublisher, stopQuotaAwarePublisher } from './services/quota-aware-publisher.js';
import { getAiQueueStatus } from './lib/ai-semaphore.js';

// — 2. Add to startup sequence —————
//
// In your existing boot sequence (where you start other services), add:
//
//   const PRIMARY_USER_ID = process.env.PRIMARY_USER_ID ?? 'your-user-id-here';
//
//   // T+25s – start prep pipelines after AI queue saturation window clears
//   setTimeout(() => {
//     startShortsPrepPipeline(PRIMARY_USER_ID);
//     startLongformPrepPipeline(PRIMARY_USER_ID);
//   }, 25_000);
//
//   // T+30s – start publisher (checks quota window, won't upload if outside window)
//   setTimeout(() => {
//     const youtubeClient = getYouTubeClient(PRIMARY_USER_ID); // your existing OAuth client
//     // getter
//     startQuotaAwarePublisher(PRIMARY_USER_ID, youtubeClient);
//   }, 30_000);

// — 3. Add to shutdown sequence —————
//
// In your graceful shutdown handler (SIGTERM/SIGINT), add:
//
//   stopShortsPrepPipeline();
//   stopLongformPrepPipeline();
//   stopQuotaAwarePublisher();

// — 4. Expose queue status in /api/health —————

```

```

//
// In your health endpoint, add the AI queue status:
//
// app.get('/api/health', (req, res) => {
//     res.json({
//         status: 'ok',
//         database: { ... },
//         memory: { ... },
//         ai_queues: getAiQueueStatus(), // ← ADD THIS
//     });
// });

// — 5. Fix infinite-evolution TikTok slot waste —————
//
// In server/services/infinite-evolution-engine.ts, find the per-platform loop
// and add a YouTube-only guard:
//
// BEFORE:
// for (const platform of platforms) {
//     await runImprovementCycle(userId, platform);
// }
//
// AFTER:
// for (const platform of platforms) {
//     if (platform !== 'youtube') {
//         log.debug(`[infinite-evolution] Skipping non-YouTube platform: ${platform}`);
//         continue;
//     }
//     await runImprovementCycle(userId, platform);
// }

// — 6. Storage methods to add —————
//
// These methods are called by the new services. Add them to server/storage.ts
// and the IStorage interface:
//
// getEncodedClipsWithoutReadyPayload(userId: string): Promise<ClipRecord[]>
// upsertShortsReadyPayload(clipId: number, payload: ShortsReadyPayload): Promise<void>
// getReadyShortsPayloads(userId: string, limit: number): Promise<ShortsReadyPayload[]>
// markShortsClipPublished(clipId: number, data: PublishedData): Promise<void>
// markShortsClipUploadFailed(clipId: number, reason: string): Promise<void>
// getClipReadStream(clipId: number): Promise<Readable>
//
// getDownloadedVideosWithoutReadyPayload(userId: string): Promise<VideoRecord[]>
// upsertLongformReadyPayload(videoId: number, payload: LongformReadyPayload): Promise<void>
// getReadyLongformPayloads(userId: string, limit: number): Promise<LongformReadyPayload[]>
// markVideoPublished(videoId: number, data: PublishedData): Promise<void>
// markVideoUploadFailed(videoId: number, reason: string): Promise<void>

```

```
//  getVideoReadStream(videoId: number): Promise<Readable>
//
//  getThumbnailReadStream(filePath: string): Promise<Readable>
//  getDailyPublishCounts(userId: string, since: Date): Promise<{ short: number; longform:
number }>
//  incrementDailyPublishCount(userId: string, type: 'short' | 'longform'): Promise<void>
//  getQuotaUsedToday(userId: string, since: Date): Promise<number>
//  recordQuotaUsage(userId: string, units: number): Promise<void>
```